

GitHub Issues Connector – Local Debugging Deep-Dive (Joe Summary)

Summary of Key Points

This document captures a multi-day debugging journey for running a Microsoft Graph External Connector (GitHub Issues Connector) locally without an Azure tenant. The final resolution mirrors the PnP sample behavior by disabling Azure-dependent components during Development while preserving production wiring.

Chronological Narrative

1. Initial failures were caused by eager construction of `ClientSecretCredential`, producing the error: `'ClientSecretCredential: tenantId is a required parameter'`.
2. Adding fake IDs or modifying `tsconfig`, ports, or `.env` files did not help because Azure Functions loads credentials during module startup, before handlers execute.
3. The decisive fix was guarding Azure Identity usage inside `graphClient.ts` using a Joe-style dev flag, returning null instead of throwing, allowing the host to continue loading functions.
4. A dev-only HTTP function (`pingJoe`) was added to verify that function discovery and HTTP routing work once startup errors are removed.
5. Subsequent `'connection refused 127.0.0.1:10000'` errors were traced to Timer triggers requiring Azure Storage. Without Azurite, these prevent the Functions host from binding to port 7071.
6. Disabling Timers in Development aligns this project with the PnP OAuth sample, which is HTTP-only.

JoeDebug Model

JoeDebug2 – Launch Azure Functions using `func.exe start`. This is the authoritative way to surface startup-time errors (auth, storage, bindings).

JoeDebug3 – Attach-only debugger on port 9229. This requires a pre-running host and was intentionally not used.

/api and Ports Explained

The `/api` route is virtual and injected by Azure Functions; no `'api'` folder should exist on disk.

Port 7071 is the default HTTP port. `'localhost refused to connect'` indicates the Functions host never finished startup (usually due to binding failures), not a routing or port flag issue.

Final State

- Graph authentication disabled safely in Development
- HTTP triggers load and are callable
- Storage-dependent timers are intentionally disabled
- Project is ready to re-enable Azure features when a tenant becomes available

References

- Microsoft Learn – Build your first Microsoft 365 Copilot Connector
- Azure Functions Node.js runtime lifecycle documentation
- PnP Copilot samples – OAuth-off Development pattern